Academic Advising Chatbot Report

Objective

The objective of this project is to develop an Academic Advising Chatbot designed to assist university business students with course information, degree requirements, instructors, schedules, and graduation planning. The system uses Retrieval-Augmented Generation (RAG) to combine a Large Language Model (LLM) with a vector database, ensuring responses are accurate, contextual, and grounded in real data from academic materials such as course catalogs, degree plans, and faculty directories. The front end of this chatbot is developed using Streamlit, offering a user-friendly web interface that allows students to interact naturally with the advising system.

Introducing RAG

Retrieval-Augmented Generation (RAG) solves these problems by combining retrieval-based search with generative AI. It has two core components:

1. Indexing Preprocesses all academic documents (DOCX, PDF, XLSX) and stores embeddings in a FAISS vector database.
2. Retrieval & Generation Fetches relevant document chunks and uses them as context for the LLM to generate grounded answers.

Technology Stack

Component	Technology	Description
Frontend	Streamlit	Interactive web-based UI for chat queries and responses
Backend	Lang Chain	Framework for chaining retrieval and generation logic
Vector Database	FAISS	Stores academic text embeddings for semantic search
LLM	Ollama (Local) or OpenAI API	Generates natural language responses
Environment Management	Python + dotenv	Securely stores configuration values

Setting up Ollama (Local Model)

Install Ollama.

Pull a local model:

- Ollama pull llama3

Add the model's name to your .env file:

- OLLAMA_MODEL=llama3

Setting Up Pinecone

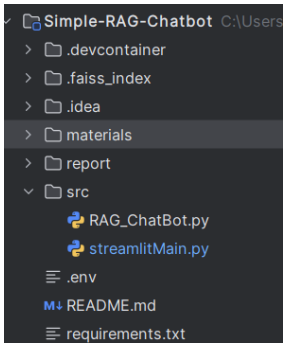
1. Sign up for a Pinecone account here.
2. After registering, create a project by filling in the Project Name, Cloud Provider, and Environment.
3. Generate an API key in the API Keys section and save it securely.
4. Add your Pinecone API key to the .env file.

Project Structure and Environment

What it is: a clear layout of files and folders plus the runtime environment (Python version and packages) required to run the project.

Why it matters: makes it easy for graders or teammates to run your code, reproduce results, and understand where each piece lives.

Suggested structure:



Data indexing

What it is: the process of reading all source files, splitting them into manageable text pieces, embedding each piece into a vector, and storing those embeddings in a vector store (FAISS or Pinecone).

Why it matters: indexing enables semantic search when a user asks a question, the retriever finds the most relevant passages quickly.

Steps:

1. Collect source files under materials/.
2. For each file: load text, clean, and optionally normalize (remove headers, unify dashes).
3. Split documents into chunks (overlap recommended).
4. Generate embeddings for each chunk.
5. Persist vectors and associated metadata to FAISS / Pinecone.

Breaking Down Text Files

What it is: loading DOCX, PDF, and Excel into plain text and splitting long texts into chunks.

Why it matters: LLMs and embedding models operate on token-limited inputs; chunking balances context scope vs. size.

Loaders to use:

- DOCX: Docx2txtLoader (langchain-community) or python-docx
- PDF: PyPDFLoader (langchain / langchain-community) or PyPDF2
- Excel: pandas.read_excel(sheet_name=None) to iterate sheets and rows

Chunking best practices:

- chunk_size 1000 characters (or tokens) works well for sentence-transformers.
- chunk_overlap 150 to keep context continuity.
- Preserve useful metadata: source filename, sheet name, page number.

Embedding Text Segments

What it is: converting each text chunk into a fixed-length numerical vector that encodes its semantic meaning.

Why it matters: vector representations allow semantic similarity search (nearest neighbors) rather than simple keyword matching.

Common embedding models:

- sentence-transformers/all-MiniLM-L6-v2 (fast, small, good quality)
- all-mpnet-base-v2 (higher accuracy, heavier)

Prompt engineering

What it is: composing a system prompt and question template that instructs the LLM how to use retrieved context and format answers.

Why it matters: good prompts prevent hallucination and give consistent, concise, and verifiable answers.

Results

The Academic Advising Chatbot successfully answers questions about course details, degree requirements, and instructor contact information. It lists core courses for each major, displays graduation requirements accurately, and provides instructors and schedules for specific terms. Using RAG ensures that the chatbot only responds using verified materials, eliminating hallucinations.

Conclusion

This project demonstrates how Retrieval-Augmented Generation (RAG) can enhance academic advising by combining the accuracy of document retrieval with the natural language fluency of modern LLMs. The Academic Advising Chatbot provides a reliable, scalable, and user-friendly solution for students.

Future Improvements:

1. Add database backend to replace Excel-based materials.
2. Implement multi-user profiles for tracking degree progress.
3. Integrate voice-based interaction for accessibility.
4. Enable automatic FAISS reindexing on new document uploads.
5. Fine-tune local LLMs for domain-specific optimization.

You can take a look at the code repository here in [my GitHub repo]

(https://github.com/vishalgoud483-tech/-Academic-Advising-Chatbot/tree/main).